esbuild and Friends

With Yarn in place to manage our third-party dependencies, we can look at a different set of problems. Problems like “How can my code consistently reference other code, when that code is in another one of my code files or in third-party modules?” and “How can I convert all of my disparate front-end assets into something that can be sent to and managed by a browser?”

All of the tools supported by jsbundling do this. These tools take our code that we have written, match up the external references, and bundle it into one (or more) files that can be sent to the browser.

We’re going to focus on esbuild. esbuild is written in Go rather than JavaScript and compiles to a native binary, which makes it very fast. The website suggests “10-100x” faster, which depends on your use case, but I will say that even on this small app, switching from webpack to esbuild took the build process from about five seconds to about a half-second—a very noticeable difference. esbuild also has a fairly powerful default set of tooling, meaning that many Rails projects can use it without further configuration. In fact, esbuild support from Rails does not even create a configuration file.

Installing esbuild

We’ve already installed esbuild via the Rails installer when we specified it as our jsbundling tool. That placed esbuild in our package.json file. You could do the same thing with yarn add esbuild.

We’ve also added a script to the package.json file that is a minimal use of esbuild.

```
% esbuild app/javascript/*.* --bundle --sourcemap --outdir=app/assets/builds
```

I’ll provide a full list of command line options in a moment, but here’s what that command does:

- It specifies the input targets as all top-level files under app/javascript/. We could limit that to explicitly specify application.js, our only matching file.